

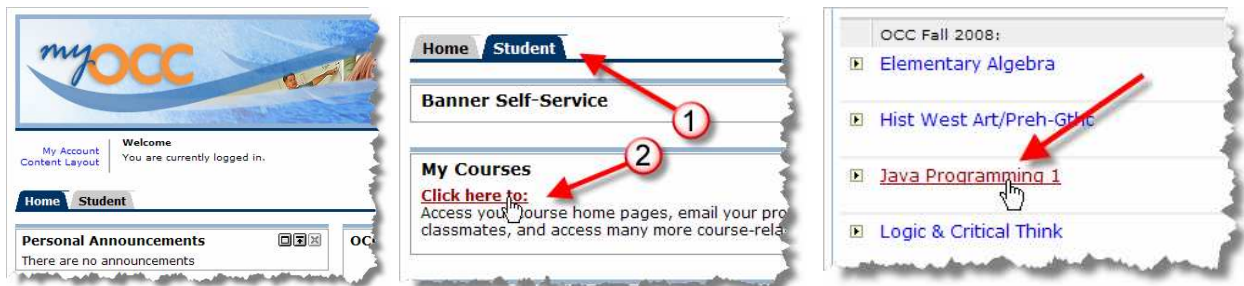
Sample Final Exams

Welcome to the CS 170 Sample Final Exams. The Final Exam is comprehensive; it will cover material from the entire semester, with an emphasis on new material. The sample exams are the same kinds of questions and the same format as the actual final exam.

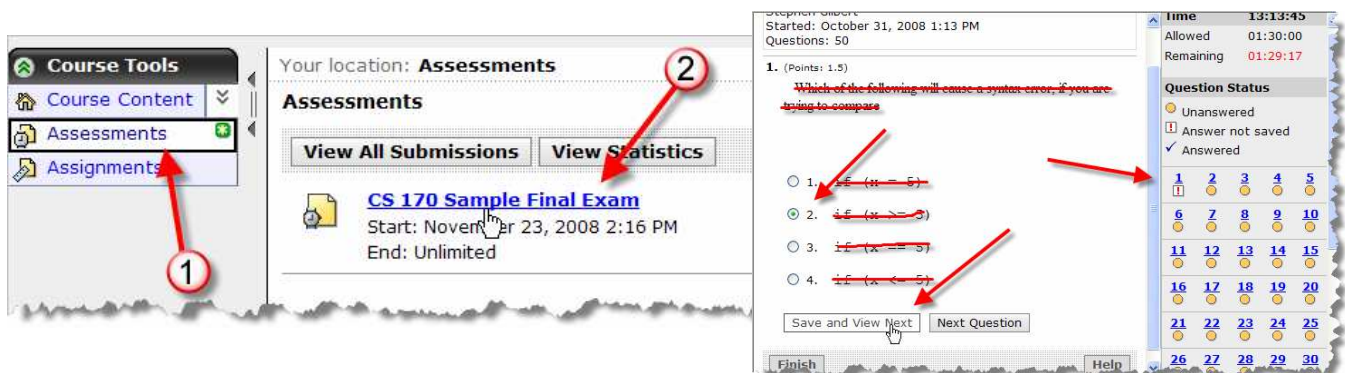
The Final exam has two parts. Part 1 is a 100-question multiple-choice exam. You'll use the Blackboard system to take both the Sample Final and the actual exam. After you've completed the Sample Final exam, you can check your score and compare your answers as an aid in preparation for the actual exam. Remember, though, that the actual exam is closed-book, but you may use two, 8.5 x 11 pages of personal notes. The second part of the exam is an open-notes, open-book programming exam. You'll use Eclipse to complete this portion of the exam, which will be graded using Web-Cat. There will be no style points for the Web-Cat scoring. The multiple-choice portion will be worth 100 points and the programming 75 points.

Part I: The Multiple-Choice Exam

Step 1: Use your MyOCC password to log into the Blackboard Vista CS 170 online classroom.

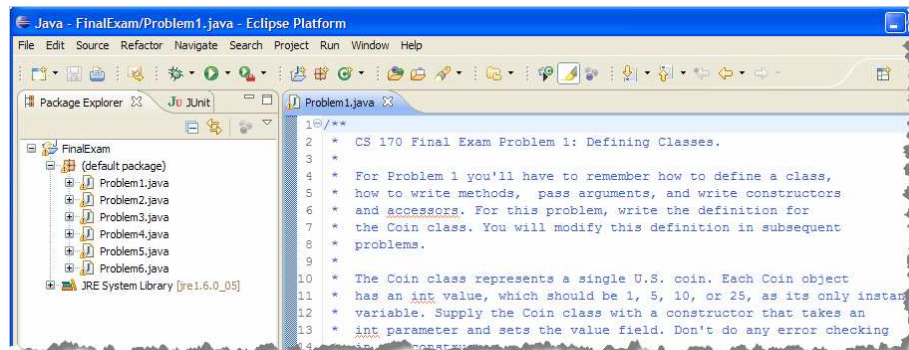


Step 2: Start the exam by clicking **Assessments** in the menu on the left. As you answer each question, make sure you click the "Save and View Next" button so that your answers are recorded. Only click the Finish button when you have completely answered all questions. You may revisit any question just by clicking its link in the table on the right. You have 90 minutes to answer all 75 questions. When you are finished, you can see your results by visiting the **Assessments** page again.



Part II: The Programming Proficiency Exam

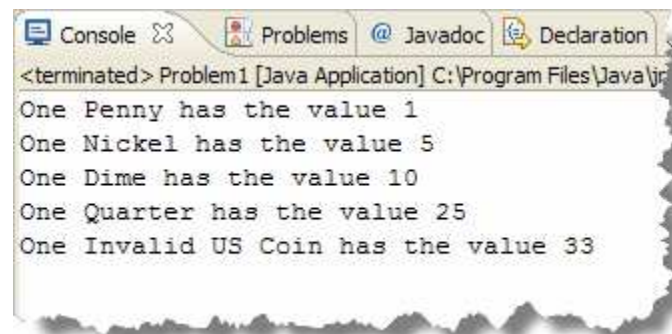
To start the sample programming exam, download the **SampleFinalProgrammingZip** file from the class Web page and store it on your local machine. Unzip the file and then Start Eclipse. From the File menu, select **Switch Workspace** and change to the **SampleFinalExam** workspace. Open the **FinalExam** project and complete the problems you find there. There you will find some sample problems with instructions written as comments at the beginning of the problem.



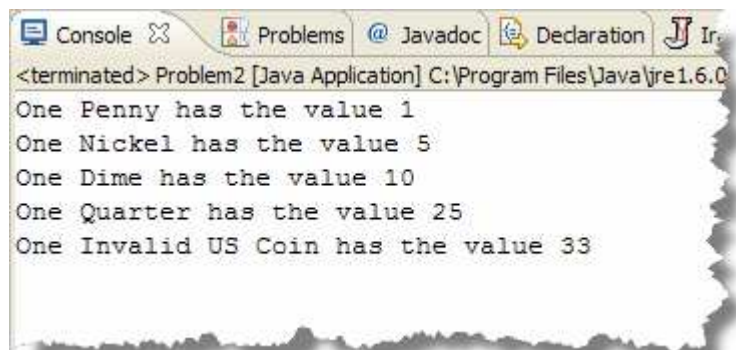
Rather than having a single large program, with lots of parts, I've divided the Final Programming problem into smaller problems that are a little more manageable.

Problem 1 - Simple Classes, Constructors and Accessors (20 Points): Define the **Coin** class, which represents a single U.S. coin. Each **Coin** object has a **value**, which must be 1, 5, 10, or 25, as its only instance variable.

Create the **Coin** class using Eclipse. Add a constructor that takes a single **int** parameter, and two methods: **getValue()** which returns the **Coin's int** value, and **toString()**, which returns the *name* of the particular **Coin** object in this exact form: "Penny", "Nickel", "Dime", or "Quarter". Make sure that the **Problem1** run method works correctly (producing the output shown here) before going on. (Grading is 50% structure, 50% correct behavior)

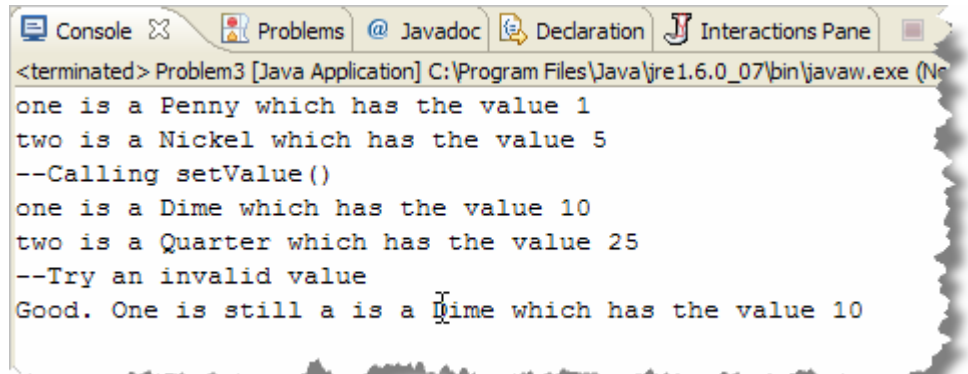


Problem 2 - Overloaded Methods or Constructors (10 points) : Add a second constructor to the **Coin** class that takes a **double** parameter. Use the parameter you receive to initialize the **value** field. You'll have to convert the parameter to an appropriate integer. For instance, the *actual* parameter .25 should be stored as 25 in the **value** instance variable. Do not add any additional instance variables. Make sure that the **Problem 2** run method works correctly.



Problem 3 - Mutator Methods and Exceptions (10 points) : Add a `setValue()` method to the `Coin` class. Your `setValue()` method should take a single `int` parameter and should use exception handling so that only valid `Coin` values can be used. You should throw an `IllegalArgumentException` when someone attempts to set an invalid `Coin` value.

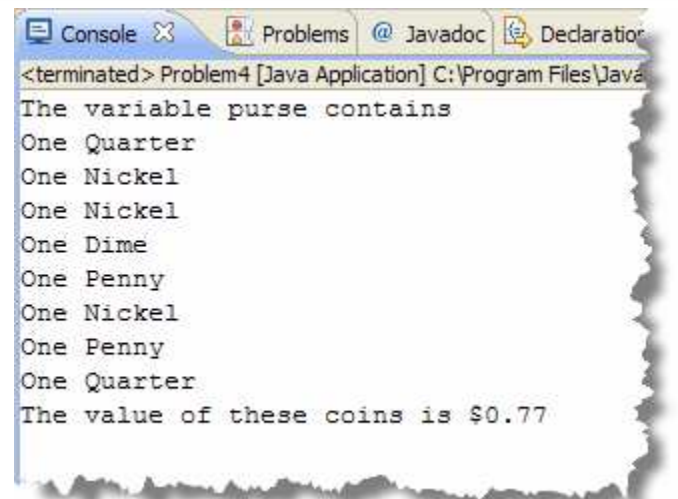
Do not modify the `Coin` constructors, so that Problem 1 and Problem 2 still work correctly, even though each of them constructs an invalid `Coin` object. Make sure that the Problem 3 run method works correctly as shown here.



```
<terminated> Problem3 [Java Application] C:\Program Files\Java\jre1.6.0_07\bin\javaw.exe (Ne
one is a Penny which has the value 1
two is a Nickel which has the value 5
--Calling setValue()
one is a Dime which has the value 10
two is a Quarter which has the value 25
--Try an invalid value
Good. One is still a is a Dime which has the value 10
```

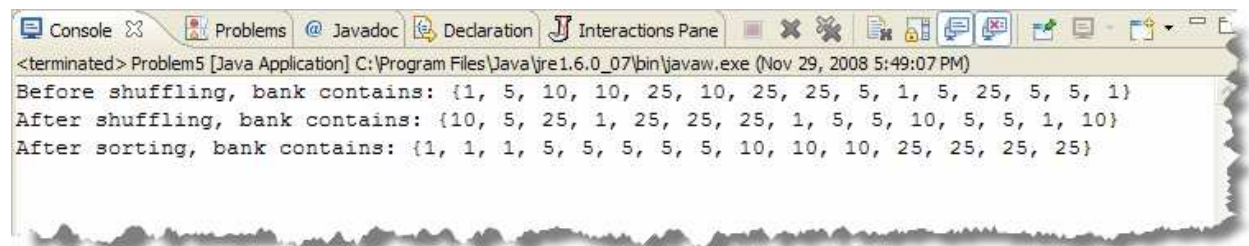
Problem 4 - Object Arrays and Standard Array Algorithms (10 points) : Write the `sum()` method (inside `Problem4.java`) that takes an array of `Coin` objects as a parameter, and returns the sum of the values that the array contains as a dollar amount. (That means if the array contains a single quarter, the method would return `.25`). Make sure that the Problem 4 run method works correctly as shown here.

For the actual exam, you should be sure you can write methods that that average (using different kinds of averaging algorithms including weighted averages, centered average, median and mode) and that make use of the algorithms for finding extreme values. I will normally give you the algorithm to use if it is anything other than a straightforward sum, average or count.



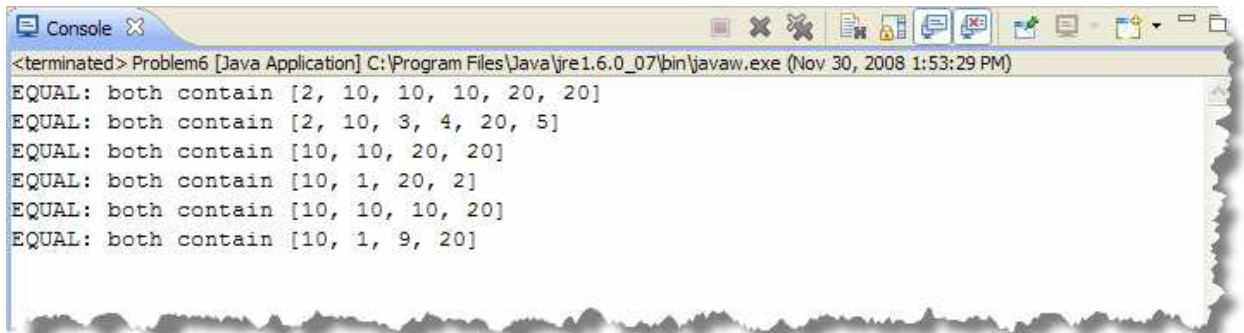
```
<terminated> Problem4 [Java Application] C:\Program Files\Java
The variable purse contains
One Quarter
One Nickel
One Nickel
One Dime
One Penny
One Nickel
One Penny
One Quarter
The value of these coins is $0.77
```

Problem 5 - Implementing the Comparable Interface (10 points) : Problem 5 first shuffles an `ArrayList` of `Coin` objects by using the `Collections.shuffle()` method and then sorts them. The `shuffle()` method works correctly, but `sort()` does not. For the sort method to work correctly, you need to modify the `Coin` class so that it implements the `Comparable<Coin>` interface. Make sure that the Problem 5 run method works correctly as shown here.



```
<terminated> Problem5 [Java Application] C:\Program Files\Java\jre1.6.0_07\bin\javaw.exe (Nov 29, 2008 5:49:07 PM)
Before shuffling, bank contains: {1, 5, 10, 10, 25, 10, 25, 25, 5, 1, 5, 25, 5, 1}
After shuffling, bank contains: {10, 5, 25, 1, 25, 25, 25, 1, 5, 5, 10, 5, 5, 1, 10}
After sorting, bank contains: {1, 1, 1, 5, 5, 5, 5, 5, 10, 10, 10, 25, 25, 25, 25}
```

Problem 6 - Array Programming with Loops (15 points) : Write the `tenRun()` method (inside `Problem6.java`) that takes an array of `int` values as a parameter, and returns a new array. For each multiple of 10 in the array parameter, change all of the values following it to be that multiple of 10, until encountering another multiple of 10. So `{2, 10, 3, 4, 20, 5}` will return a new array with the value `{2, 10, 10, 10, 20, 20}`. Make sure that the `Problem6` run method works correctly as shown here. (This problem will be one of the JavaBat Array2 exercises.)



```
<terminated> Problem6 [Java Application] C:\Program Files\Java\jre1.6.0_07\bin\javaw.exe (Nov 30, 2008 1:53:29 PM)
EQUAL: both contain [2, 10, 10, 10, 20, 20]
EQUAL: both contain [2, 10, 3, 4, 20, 5]
EQUAL: both contain [10, 10, 20, 20]
EQUAL: both contain [10, 1, 20, 2]
EQUAL: both contain [10, 10, 10, 20]
EQUAL: both contain [10, 1, 9, 20]
```

Submitting the Programming Exam

Save your work and **submit** the assignment. Choose **Project Submit Project** and select the appropriate exam. Fill in your Web-Cat user ID and password. This will be **different** than your MyOCC user ID and password used to log into BBVista. If you have problems submitting your exam, tell me as soon as possible.

Do the easy problems first; submit often.